Session 2 — Foundations 2

HPC & SLURM (getting onto the cluster)

Qiuyu Yu, Human Development and Family Science (HDFS), Auburn University

Recap: Session 1 gave you the Linux command line: paths, scripts, permissions, redirection.
Goal today: log in to the cluster, move your data around, and submit and monitor a real job.

Roadmap for today

1. **Connecting to your HPC** (login vs compute, how a job runs)
2. **Storage, modules,** and moving data
3. **SSH & the web portal** (keys, what SSH really is, how interactive apps work)
4. **Editing & running scripts**
5. **Submitting, monitoring & debugging jobs** (SLURM, arrays, GPUs, parallelism, reading logs)

Next session we set up the developer tools (Git, VS Code, Python, containers, Jupyter).

3. Connecting to your HPC

Names below are GACRC/Sapelo2 examples; your site's portal, scheduler, and storage may differ.

```
ssh user@cluster.university.edu   # log in (use the VPN off-campus)
```

- **One rule first:** the login node is only for logging in and submitting, **not** for heavy compute.
    - heavy work goes to an interactive compute node or a submitted job. Administrators will usually tell you not to load any modules or run analysis scripts on the login node.

3. Connecting to your HPC (cont.)

- Off-campus usually needs the **VPN**; many sites also require **two-factor auth** (Duo or an OTP app) on top of your password.
- Other ways in: a web portal (e.g. Open OnDemand).
- A remote desktop / interactive app gives you a GUI when you need one.

3a. What an HPC really is

- Not one super-computer, but **many ordinary computers (nodes)** networked together and **shared** among many users.
- **Login node:** where you land when you connect. Small, shared, for editing and submitting, *not* for heavy compute.
- **Compute nodes:** where real work runs. You do not log into them directly; you **request** them.
- **Shared filesystem:** every node sees the same files, so your data and scripts are available everywhere.
- A **scheduler** (SLURM) is the traffic controller that hands out compute nodes fairly.

3a2. How your job actually runs

1. Write a script with `#SBATCH` resource requests on top.
2. `sbatch job.sh` puts it in a **queue**.
3. The scheduler waits until a compute node with the requested resources is free.
4. It runs your script **on that compute node** (not the login node).
5. Output goes to your log files; when it finishes, the node is freed for the next user.

```
squeue --me             # check queued / running status
seff <jobID>            # afterwards: how much it actually used
```

The payoff: 100 subjects on a laptop would take days; an HPC runs them in **parallel**, one slice per subject.

3b. Storage map (know before you write data)

Typical tiers (names and policies vary by institution):

Space	Use	Watch out
Home	scripts, small "permanent" files	tiny quota
Work	shared lab space	store *processed* data, don't compute here
Scratch	fast working space	may be purged (e.g. after 30 days)
Project	archive	may need a transfer tool (e.g. Globus)

Know where your data lives, and whether it will still be there next month.

3c. Loading software (the module system)

On a cluster, software is not installed globally; you load it per session.

```
module avail              # what is available
module spider AFNI        # search for a specific tool
ml AFNI/24.3.06           # load it  (ml = module load)
module list               # what is loaded right now
module purge              # unload everything
```

- Loading a module puts the tool on your `PATH` so the command is found.
- "command not found" on a cluster usually means the module is not loaded.

3d. Moving data in and out

```
scp file user@cluster:/scratch/you/      # copy one file up
scp -r dir/ user@cluster:/scratch/you/   # copy a folder up
rsync -a --info=progress2 src/ dest/     # resumable, only changed files
```

- `rsync` is the main tool: safe to re-run, copies only what changed.
- Large transfers or archive tiers: a GUI / transfer tool (e.g. FileZilla, Globus).
- Big transfers: use your site's **transfer / data nodes** (not the login node), and avoid the VPN, which throttles bandwidth.
- Many small files move slowly; `tar` them into one archive first (`pigz` for fast parallel compression).

3e. SSH keys (log in without a password)

A key pair = a **private key** (stays on your computer) + a **public key** (you hand to the server).

```
ssh-keygen -t ed25519            # make a key pair (once)
ssh-copy-id user@cluster         # install your public key on the HPC
ssh user@cluster                 # now logs in with no password
```

- Same idea for **GitHub**: add your public key in *Settings* -> *SSH keys*, then `git push` needs no password.
- Never share or commit your **private** key.

3f. What SSH really is

- **SSH = Secure Shell:** an **encrypted tunnel** between your computer and the server, and inside it, a command line running **on the remote machine**.
- It just **relays keystrokes and output**: you type locally, it runs remotely, output comes back. Your terminal only *feels* like it is on the server.
- **Everything is encrypted:** keystrokes, output, passwords. (It replaced the old plaintext `telnet`.)

3f2. SSH: keys, and what rides on it

- **Key-pair login** (slide 3e): the server sends a challenge, your computer signs it with the **private key**, the server verifies with your **public key**. You prove you hold the key **without ever sending it**.
- SSH is also a **general-purpose tunnel** that other tools ride on: `scp` / `rsync` / `sftp`, **VS Code Remote-SSH**, and **port forwarding** (how a browser reaches Jupyter on a compute node).

3g. The web portal & interactive apps

A portal like **Open OnDemand** is a **website in front of the cluster**: a shell, a file browser, and "interactive apps" (Jupyter, RStudio, a **Desktop**).

When you launch an interactive app, the portal:

1. **Submits a SLURM job for you**, with the resources you picked (cores, mem, time, partition).
2. The job lands on a **compute node**: not the login node; that is the point, you get real compute.
3. The job **starts a server** there: Jupyter starts `jupyter`; Desktop starts a **VNC** desktop.
4. The portal **proxies** your browser to that server's port on the compute node.

3g. The web portal & interactive apps (cont.)

- So the desktop / notebook you see **is a program running inside your job on a compute node.**
- It **uses your walltime**: when the time runs out, the job is killed, and your desktop with it.

3g2. Three ways to get a remote GUI

People mix these up; they are different mechanisms:

Way	What travels the network	Runs where
X11 forwarding (`ssh -X`)	drawing commands	wherever you ssh'd (often the login node)
VNC desktop (portal "Desktop")	screen pixels	a compute-node job
Web app (Jupyter / RStudio)	the app's web page	a compute-node job

3g3. X11 vs VNC, in practice

- **X11** forwards **drawing commands**: the app runs on the remote, but its window is rendered by an **X server on your own machine** (that is why your local computer is the "server", not the cluster).
- That local X server is built in on Linux, but you install one on **Mac (XQuartz)** or **Windows (VcXsrv / Xming)**. X11 is simple but laggy over distance, and runs where you logged in (keep it off heavy compute).

3g3. X11 vs VNC, in practice (cont.)

- **VNC / web apps** run inside a real job on a compute node, so the app keeps running if your connection drops and you can reconnect (unlike a raw `ssh -X` session).
- But **VNC streams pixels**, so how smooth the desktop feels depends on your **network**: a slow or high-latency link lags, worst when much of the screen changes (scrolling, 3D volume rendering). The heavy compute stays on the node; only the display is network-bound.

3h. When SSH / the terminal wins

Reach for SSH / the terminal when you need to:

- **Script & automate:** loop over subjects, submit and monitor many jobs, and a browser can't.
- **Move data:** `scp` / `rsync` for large or many files (resumable); portal upload is clunky at scale.
- **Edit on the cluster:** VS Code **Remote-SSH** needs SSH, not the portal.
- **Stay reproducible:** every step is a typed command you can save and rerun; a GUI click leaves no record.
- **Work light / on a bad line:** a terminal is instant; no pixel streaming. Add `tmux` to survive disconnects.

3h2. When the portal / desktop wins

Reach for the portal / desktop when you want:

- To **eyeball MRI images**: registration, QC, FSLeyes / AFNI / SUMA viewers. Visual checks are the natural home of a GUI desktop.
- An **interactive Jupyter / RStudio**, or a quick file peek / drag-and-drop, with no setup and no SSH keys.

Not either/or: you *can* reach a desktop over SSH too, by tunneling the VNC/X11 port through the SSH connection (VS Code can forward it for you). But it is extra setup, and campus **VPN / 2FA / jump-hosts** make it fiddlier. For just looking at images, the portal's **Desktop** is usually the path of least resistance.

4. Editing & running scripts

```
nano script.sh        # Ctrl+O save, Ctrl+X exit; arrow keys move the cursor
vim script.sh         # press i to insert, Esc, then :wq to save & quit
dos2unix script.sh    # fix Windows line endings (\r) if it won't run
chmod +x script.sh    # make it executable
./script.sh /path/arg
```

- Easiest for beginners: edit in **VS Code** (often available via a web portal), `nano` for quick fixes. (VS Code setup is Session 3.)

5. Submitting a job (SLURM)

A job script is a normal shell script with `#SBATCH` resource requests on top:

```
#!/bin/bash
#SBATCH --job-name=test
#SBATCH --cpus-per-task=4
#SBATCH --mem=8gb
#SBATCH --time=01:00:00
#SBATCH --output=log_%j.out

echo "running on $(hostname)"
```

Commands here are SLURM; some sites use PBS/SGE (different commands, same ideas).

5a. Requesting resources (what to ask for)

```
#SBATCH --cpus-per-task=4     # CPU cores (parallel threads)
#SBATCH --mem=8gb             # RAM (Tip: ALWAYS specify 'gb'. Omitting it defaults to MB, crashing your job instantly)
#SBATCH --time=01:00:00       # Walltime Max (automatically killed if exceeded)
#SBATCH --gres=gpu:1          # Request 1 GPU (only use when necessary)
```

- Ask for what you need, not the maximum; bigger requests wait longer in the queue.
- Right-size with `seff <jobID>` after a test run, then adjust.
- Set memory with **either** `--mem` (per node) **or** `--mem-per-cpu`, not both.
- More cores only help if the tool is actually parallel (Session 4).
    - If you request 4 cores, match your code parameters (`threads=4` or `n_jobs=4`), otherwise 3 cores sit idle and waste your allocation.

5b. CPU vs GPU (and when GPU helps)

- **CPU:** a few **powerful, flexible** cores. Great at varied, sequential, branchy work, like a few expert chefs who can cook anything.
- **GPU: thousands of small** cores doing the **same simple operation on lots of data at once**, like an army of line cooks doing one step in parallel.
- **GPU wins** when the work is massively parallel and uniform (big matrix math): deep learning, some registration, diffusion (`eddy`), DL denoising/segmentation.
- **CPU wins** for branchy, mixed, sequential work, which is most classic fMRI preprocessing (AFNI, fMRIPrep).
- On HPC, request a GPU explicitly (`--gres=gpu:1`), only when your tool actually uses one, or it is wasted.

5c. Parallelism: just the intuition

- Many neuroimaging tools are mostly **single-threaded** and use **OpenMP** (many cores, one node).
- One subject: few tasks, more CPUs per task. Many subjects: an **array job**.
- **So in practice:** one node, `--ntasks=1`, several `--cpus-per-task` (OpenMP threads); scale out with **array jobs**, not multiple nodes. MPI / multi-node is rare in fMRI.
- Analogy: OpenMP = one crew sharing tools; MPI = separate crews radioing each other.
- Rule of thumb: more cores is not faster unless the tool scales. Test small first.

5d. Array jobs

One task per subject, all queued from a single script and run in parallel:

```
#SBATCH --array=1-100%10        # 100 tasks, at most 10 running at once
#SBATCH --output=log_%A_%a.out  # %A = array job ID, %a = task index (avoid one shared log)
```

- **Map the task number to a subject** with a list file (the core pattern):

```
subj=$(sed -n "${SLURM_ARRAY_TASK_ID}p" subjects.txt)   # take line N of the list
```

- Re-run only the ones that failed: `#SBATCH --array=3,7,12`.
- The `#SBATCH` cpus / mem / time are **per task**, not for the whole array.

5d2. GPUs

GPU (specialized, restricted, expensive):

- Often needs a specific **partition / account**, so do not request one blindly. Ask for a **specific type** so you do not land on an incompatible card: `--gres=gpu:a100:1`.
- **Requesting a GPU is not using one:** you still need the GPU build of the tool plus the right module; confirm with `nvidia-smi` inside the job. In a container, add `--nv` (Singularity / Apptainer).
- Worth it only for GPU-designed work (deep learning: PyTorch/TensorFlow, accelerated molecular dynamics), where it finishes far faster than a CPU.

5d3. Job dependencies (chaining a workflow)

Job dependency: run Job B only after Job A succeeds, so a failed preprocessing step does not waste cluster time.

- Types: `afterok` (A succeeded), `afterany` (A ended either way, good for cleanup), `afternotok` (A failed).
- Build the chain in a script with `--parsable` (it returns just the numeric ID):

```
jid=$(sbatch --parsable step1.sh)
sbatch --dependency=afterok:$jid step2.sh
```

5e. SLURM placeholders & variables

- **Job-level auto-placeholders:** SLURM fills these into filenames at run time (like the `%j` in `--output=log_%j.out` earlier).
  - `%j` -> unique numeric job ID (e.g. `slurm-%j.out`)
  - `%x` -> job name (e.g. `simulation-%x.out`)
  - `%a` -> array task index, for batch processing
  - `%u` -> your username (handy for shared scratch logs)

5e. SLURM placeholders & variables (cont.)

- **System-level variables:** predefined environment variables, referenced with a leading `$`.
  - `$USER` & `$HOME` -> your username and home path
  - `$PWD` -> absolute path of the current working directory
  - `$SLURM_CPUS_PER_TASK` -> matches the CPU cores you requested

5f. Submitting & monitoring

```
sbatch job.sh          # submit the job
squeue --me            # my queued / running jobs
scancel <jobID>        # cancel a job
sacct -X               # history of finished jobs
seff <jobID>           # how much memory / CPU it actually used
```

- `tmux` / `screen`: keep a long interactive session alive if your connection drops.
- A job killed at its walltime does not restart by default; add `#SBATCH --requeue` only if your tool can resume from a checkpoint.

5f2. Interactive jobs (test before you batch)

Grab a compute node and work on it live: ideal for debugging a script or trying a command before committing to a big batch job.

```
srun --pty --cpus-per-task=4 --mem=8gb --time=02:00:00 bash   # a shell on a compute node
salloc --cpus-per-task=4 --mem=8gb --time=02:00:00            # allocate a node, then work on it
```

- Some sites wrap this in a helper (e.g. `interact -c 4 --mem 8G`); check your docs.
- Add `--x11` (with `ssh -X`) to run a GUI viewer on the node.
- It still spends your allocation and walltime, so exit when done to free the node.
- Rule of thumb: **test small interactively, then submit the full run with `sbatch`.**

5f3. When a job fails: read the log first

- Every job writes a log (`slurm-%j.out` , or your `--output` file) in the submit directory. **Read it, top to bottom.** The real cause is usually at the **bottom of the error trace**, and often something simple.
- **stdout vs stderr:** normal output and errors both land there by default; while testing, split them with `2> err.log` (Session 1) to isolate the failure.
- **Why did it die?** `seff <jobID>` and `sacct -j <jobID>` show the **exit code** and status:
  - `OUT_OF_MEMORY` or "Killed" -> raise `--mem` .
  - `TIMEOUT` -> raise `--time` .
- **Reproduce it interactively** (slide 5f2): load your modules and run the steps one at a time to see exactly where it breaks.

5f4. Most failures are one of these

- **Wrong path** (the #1 cause): the file is not where you think, or a typo. Linux is **case-sensitive**, so `Sub01` and `sub01` are two different names. Check `pwd`, and use absolute paths in job scripts.
- **Module / version:** you forgot `ml`, or a new version changed the syntax. `module list` shows what is loaded; pin the version (`ml AFNI/24.3.06`).
- **Line endings:** a script edited on Windows carries `\r`; `file script.sh` reports "CRLF", fix it with `dos2unix`.
- **Permissions:** "Permission denied" means `chmod +x` the script (Session 1).
- **The tool has a bug, or your inputs break its assumptions:** paste the exact error into a search engine, check `--help` and the tool's issue tracker.

(fMRI-pipeline-specific troubleshooting comes in Session 8.)

5g. Processes: foreground, background, kill

A running command is a **process**. By default it runs in the **foreground** and ties up your terminal.

```
mycmd &           # run it in the background without generating log in the terminal
Ctrl+C            # stop the foreground command
Ctrl+Z , then bg # pause it, then resume in the background
top   /   htop    # see what is running, and its CPU / memory
kill <PID>        # stop a process by its ID
```

- On an HPC, your real work runs as a scheduler-managed process on a compute node.

Assignment (Session 2)

1. Log into your HPC, open a terminal, find your home / scratch space
2. Move a small file up to scratch with `scp` or `rsync`
3. Set up an SSH key so you can log in without a password
4. Load a module (`ml` / `module load`) and confirm the tool is on your `PATH`
5. Write & submit a tiny job with `sbatch`; check its status (`squeue --me`) and resource usage (`seff`)

Reading: https://qiuyuyu3.github.io/fMRI-Data-Processing-Manual-on-HPC/introduction/

You can now...

- get onto the HPC and not run heavy jobs on the login node
- load modules, move data, and log in with an SSH key
- submit a job, monitor it, and read how much it actually used

Next: set up your developer tools: Git, VS Code, Python environments, containers, and Jupyter.